CIKLUSOK (ITERÁCIÓ) – A WHILE CIKLUS

Az iteráció röviden azt jelenti, hogy a program ugyanazt a kódblokkot újra és újra végrehajtja, a feladattól függő számban, vagy egy logikai feltétel bekövetkeztéig. Az iterációt végrehajtó programozási struktúrát ciklusnak, huroknak nevezzük. A programozásban az iterációnak kétféle típusa van: határozatlan és határozott.

Határozatlan iteráció: a ciklus végrehajtásának száma nincs pontosan előre megadva. A kijelölt blokkot a program többször végrehajtja, mindaddig, amíg bizonyos feltétel(ek) teljesülnek.

Határozott iteráció: a kijelölt programblokk végrehajtásának száma egyértelműen meghatározott már akkor, amikor a ciklus elindul.

A WHILE CIKLUS

A Python „while” ciklus felépítése nagyon egyszerű:

```
1. while <expr>:
2. <statement(s)>
```

A „statements” rész tulajdonképpen a létrehozott ciklus magja, az a programblokk, ami meghatározott számban lefut, az „expr” részben szabjuk meg, hogy ez hányszor történjen meg.

Nézzünk egy egyszerű példaprogramot:

```
1. n = 5
2. while n > 0:
3. n -= 1
4. print (n)
```

Az n értéke kezdetben 5. A 2. sorban a while állítás vizsgálatána eredménye(n> 0), ami igaz, tehát a ciklusmag lefut. A 3. sorban a ciklusmagban n-t 1-el csökkentjük, majd az eredményt kinyomtatjuk.

Amikor a ciklusmaglefutott, a program végrehajtása visszatér a hurok tetejére a 2. sorba, és a kifejezést újra kiértékeljük. Még mindig igaz, tehát a ciklusmag újra végrehajtja a feladatot, és az eredmény nyomtatásra kerül.

Ez addig folytatódik, amíg n nullává nem válik. Ezen a ponton, amikor a kifejezést teszteljük, az eredmény hamis, így a hurok véget ér.

Fontos, hogy a while hurok a vezérlő logikai kifejezést előbb teszteli, mielőtt bármi más megtörténik. Ha hamis a kiértékelés eredménye, akkor a ciklusmag egyszer sem fut le.

A ciklusmagban szereplő n -= 1 kifejezés jelentése: n értékét csökkentsed eggyel.

```
5. n = 0
6. while n > 0:
7. n -= 1
8. print (n)
```

A fenti program az előzőek értelmében egyszer sem fut le, mert a kiindulási feltétel (n > 0) hamis.

Nézzünk egy példát „felfelé” számlálásra, amikor a ciklusváltozó értékét egyesével növeljük!

```
1. count = 0
2. while (count < 9):
3. print ('The count is:', count)
4. count = count + 1
5.
6. print ("Good bye!")
```

Az első sorban a count változó értékét nullára állítjuk, a while állítás vizsgálatát pedig úgy állítjuk be, hogy a vizsgálat mindaddig igaz legyen, míg a változó kisebb mint 9.

A VÉGTELEN CIKLUS

A ciklus végtelen ciklussá válik, ha avizsgálati részben beállított feltétel soha nem válik hamissá(FALSE). Ezzel a megoldással azonban óvatosan kell eljárni a ciklusok programozása során, mert olyan ciklust állíthatunk elő, amiből a programunk nem tud kilépni.Az ilyen ciklust végtelen ciklusnak hívják.

```
1. var = 1
2. while var == 1 : # This constructs an infinite loop
3. num = int(input("Enter a number :"))
4. print ("You entered: ", num)
5.
6. print ("Good bye!")
```

A második sorban beállított logikai feltétel állandóan igaz értéket ad, így a ciklus végtelenné válik. A ciklusból való kilépés egyetlen módja a Ctrl + c billentyűkombináció alkalmazása.

Elegánsabb megoldás, ha figyeljük a bevitt értékeket, és egy előre megadott szám bevitelekor (pl. 0) a „break” paranccsal kilépünk a ciklusból:

```
1. while True: # This constructs an infinite loop
2. num = int(input("Enter a number :"))
3. print ("You entered: ", num)
4. if num == 0:
5. break # break here
6.
7. print ("Good bye!")
```

Hasonló megoldást eredményez a cikluson belül elhelyezett felhasználói döntést kérő vezérlésátadó utasítás, aminek segítségével elegánsan tudunk kilépni a programból:

```
1. import random
2.
3. while True:
4. input("Press enter to roll the dice")
5.
6. # get a number between 1 to 6
7. num = random.randint(1,6)
8. print("You got",num)
9. option = input("Roll again?(y/n) ")
10.
11.# condition
12.if option == 'n':
13.break
```

A példa kockadobást szimulál. A program első sora importálja a véletlenszám generáláshoz szükséges library-t, ennek a segítségével állítjuk elő az 1-6 közé eső egész számot, ami egy „whileTrue” típusú végtelen cikluson belül mindaddig fut, amíg az „új dobás?” (Roll again?) kérdésre y(es) a válasz. Nem válasz esetén kilépünk a ciklusból.

A Pythonban a for utasítás használata, működése kissé különbözik attól, amit a C vagy a Pascal nyelvben megszokhattunk. Ahelyett, hogy egy változó mindig a számok aritmetikai növekedésével iterálódna (mint például a Pascal-ban), vagy, hogy a felhasználó képes legyen meghatározni mind az iterációs lépést, mind a megállási feltételt (C-nyelv), a Python „for” parancsa bármilyen sorozat (lista vagy akár karakterlánc) elemeit annak sorrendjében tudja „bejárni”. Például:

```
1. words = ['macska', 'kutya', 'pirosalma']
2. for w in words:
3. print(w, len(w))
```

A program végigmegy a lista elemein, (macska, kutya, pirosalma), egymás után kinyomtatja az elemeket, illetve minden egyes szó mellé kiírja, hogy hány karakterből állnak. Ezt a „len()” függvénnyel oldja meg, ami az utána zárójelben megadott string karakterszámát adja.

```
1. >>> len("kutya")
2. 5
```

A „RANGE()” FÜGGVÉNY

Amennyiben az adataink szekvenciálisak, és az iterációt ezen a sorozaton kell végrehajtani, jó szolgálatot tesz a „range()” függvény.

```
1. for i in range(5):
2. print(i)
```

Az adott végpont soha nem része a generált sorozatnak. A (10) tartomány 10 értéket generál, a 10-es érték nem lesz a sorozat eleme. Lehetőség van arra, hogy a tartomány egy másik számtól induljon, vagy akár megadhatunk 1-től eltérő növekményt is, ez akár negatív is lehet! A növekményt sokszor lépés-nek hívjuk.
range(kezdőérték, végérték, lépés)

```
1. range(5, 10)
2. 5, 6, 7, 8, 9
3.
4. range(0, 10, 3)
5. 0, 3, 6, 9
6.
7. range(-10, -100, -30)
8. -10, -40, -70
```

A sorozat elemein történő iteráláshoz a következőképpen kombinálhatjuk a „range ()” és a „len ()” függvényeket:

```
1. a = ['Mary', 'had', 'a', 'little', 'lamb']
2. for i in range(len(a)):
3. print(i, a[i])
```

BEÁGYAZOTT CIKLUSOK

A beágyazott hurok egy hurok belsejében lévő hurok. A “belső hurok” végrehajtása egyszer történik a “külső hurok” minden egyes iterációja esetén:

```
1. type = ["red", "big", "tasty"]
2. fruits = ["apple", "banana", "cherry"]
3.
```

```
4. for x in type:
5. for y in fruits:
6. print(x, y)
```

LISTA ALKALMAZÁSA A FOR CIKLUSBAN

A Python lista tulajdonképpen elemek, adatok felsorolása vesszővel elválasztva. Mondhatjuk azt is, hogy tulajdonképpen egy adattömb, ezekről a későbbikben lesz szó. A lista elemei lehetnek karakterek, stringek, számok. Egy lista elemeit könnyen ki tudjuk íratni egy „for” ciklus segítségével:

```
1. mylist = ["apple", "banana", "cherry"]
2. for x in mylist:
3. print(x)
```

Amennyiben a lista pl. számokat tartalmaz:

```
1. mylist = [13, 22, 44, 55]
2. for x in mylist:
3. print(x)
```

Megfigyelhető, hogy a „for” ciklus és a lista kombinálásával kinyomtatott számsor elemei között nem egyenletes a növekmény.
A lista elemeinek egymás után történő kinyomtatása megoldható a „for” ciklus és a „range()” függvény használatával:

```
1. mylist = [13, 22, 44, 55, 65, 92]
2. for x in range(2,5):
3. print(mylist[x])
```